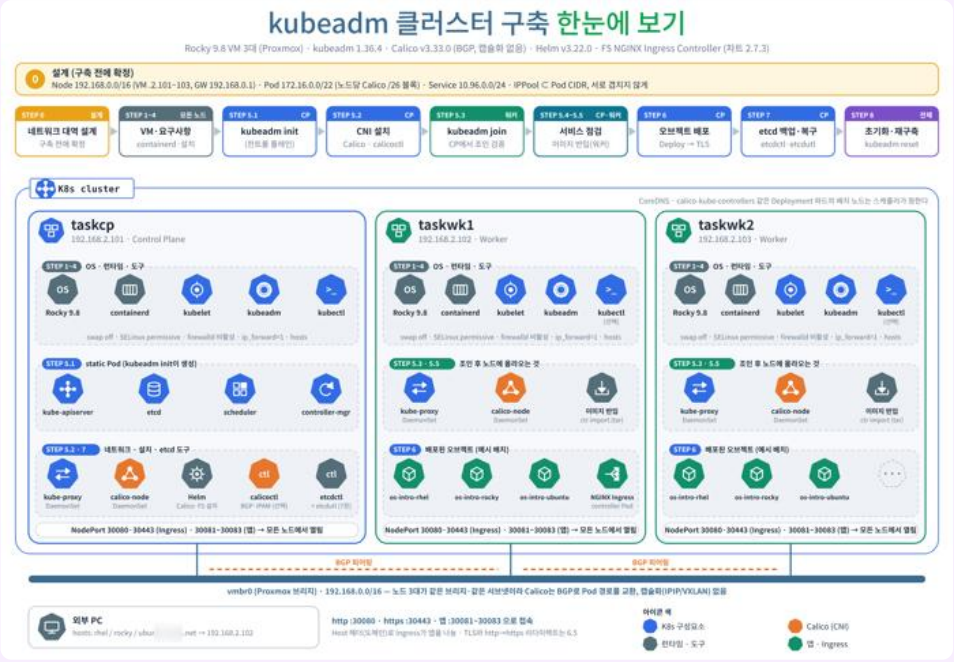
kubeadm 클러스터 구축 가이드

구축 스펙

항목	값
OS	Rocky Linux 9.8
노드	taskcp(CP) 192.168.2.101 / taskwk1 192.168.2.102 / taskwk2 192.168.2.103 (모두 /16, GW 192.168.0.1)
Kubernetes	kubeadm, kubelet, kubectl 1.36.4 (패키지 버전 고정)
컨테이너 런타임	containerd (containerd , Docker 저장소)
CNI	Calico v3.33.0, Helm 설치, BGP, 캡슐화 없음
Helm	v3.22.0
Pod CIDR / Service CIDR	172.16.0.0/22 / 10.96.0.0/24
NodePort	앱 30081(rhel) 30082(rocky) 30083(ubuntu), Ingress 컨트롤러 30080(http) 30443(https)
Ingress 컨트롤러	F5 NGINX Ingress Controller, 차트 2.7.3 (앱 5.6.3), Helm 설치
앱 이미지	os-intro-rhel:1.0 / os-intro-rocky:1.0 / os-intro-ubuntu:1.0 (레지스트리 없이 노드에 직접 import)

진행 순서와 실행 위치

```
[설계]      0 네트워크 대역 설계 (구축 전에 확정)
[모든 노드] 1 VM 세팅 → 2 요구사항 → 3 containerd → 4 kubeadm/kubelet/kubectl
[CP]        5.1 kubeadm init → 5.2 CNI(NetworkManager, Helm, Calico, calicoctl)
[워커]      5.3 사전 세팅 확인 → kubeadm join   /  [CP] 조인 검증
[CP/워커]   5.4 클러스터 서비스 점검 → 5.5 이미지 반입(워커)
[CP]        6 오브젝트: Deployment → Service → Ingress 컨트롤러 → Ingress → TLS
[CP]        7 etcd 백업과 복구
[전체]      8 초기화와 재구축
```



0. 네트워크 대역 설계
1. VM 세팅
    1.1 네트워크 검증
    1.2 hosts 등록
    1.3 외부 repository 접근성 확인
    1.4 스냅샷(선택)
2. kubeadm 설치 요구사항
    2.1 OS 버전 확인
    2.2 모든 노드에서 MAC 주소와 product_uuid가 고유한지 확인
    2.3 네트워크 어댑터 확인
    2.4 필수 포트 확인
    2.5 스왑 구성
    2.6 SELinux permissive 설정
    2.7 IPv4 포워딩
3. 컨테이너 런타임 설치
4. kubeadm, kubelet, kubectl 설치
    4.1 yum repository 추가
    4.2 버전 확인 후 설치
    4.3 버전 고정 확인
5. 클러스터 세팅
    5.1 컨트롤플레인 구성: kubeadm init
    5.2 네트워크 서비스: CNI
        (1) NetworkManager 설정
        (2) Helm 설치
        (3) Calico 설치
        (4) 검증
        (5) calicoctl 설치(선택)
    5.3 워커 노드 조인
        (1) 노드별 사전 점검
        (2) join 실행
        (3) join 검증
    5.4 클러스터 서비스 상태 점검
    5.5 이미지 반입
6. 클러스터 오브젝트
    6.1 Deployment와 probe

6.2 Service
6.3 Ingress Controller
6.4 Ingress 리소스: 호스트 기반 라우팅
6.5 TLS와 http→https 리다이렉트
7. etcd 백업 및 복구
7.1 백업
(1) etcdctl/etcdutl 설치
(2) 환경변수 설정
(3) 기존 상태 확인
(4) 스냅샷 저장과 검증, 복사
7.2 복구
(1) 컨트롤 플레인 중지
(2) 복구 시작
(3) etcd.yaml의 호스트 경로만 수정
(4) 컨트롤 플레인 복귀 확인
(5) kubelet 재시작
7.3 롤백
8. 초기화와 재구축

---

0. 네트워크 대역 설계

대역이 겹치면 안 되는 이유, 선정 근거

등을 참고하여 Pod CIDR, Service CIDR, 사무실 대역이 겹치지 않게 네트워크 대역을 설계한다.

설계한 대역을 아키텍처로 나타낸다.

1. VM 세팅

1.1 네트워크 검증

호스트네임과 ip(VM IP, 게이트웨이)를 확인한다.

```
hostnamectl                             # Static hostname: taskcp / taskwk1 / taskwk2
ip -br a                                # 인터페이스명(IF)과 IP/16 확인 — IF 명을 메모
ip r                                    # default via 192.168.0.1, 192.168.0.0/16 경로
```

고정 IP가 필요하다면 nmcli나 nmtui로 IP 설정을 바꾼다.

```
sudo nmcli con mod "<인터페이스명>" ipv4.method manual \
  ipv4.addresses <바꿀IP/바꿀대역> ipv4.gateway 192.168.0.1 ipv4.dns "<DNS주소>"
sudo nmcli con up "<인터페이스명>"
sudo hostnamectl set-hostname taskcp
```

DNS 주소는 `cat /etc/resolv.conf` 후 nameserver 뒤에 있는 주소들을 넣어준다.

1.2 hosts 등록

각 VM들이 이름으로 통신하기 위해 /etc/hosts에 등록해 준다.

```
sudo tee -a /etc/hosts <<'EOF'
192.168.2.101 taskcp
```

```
192.168.2.102 taskwk1
192.168.2.103 taskwk2
EOF
getent hosts taskwk1   # 확인
```

1.3 외부 repository 접근성 확인

설치하기 전 패키지 저장소 등에 접근할 수 있는지 미리 확인한다.

```
curl -sI https://pkgs.k8s.io        | head -1
curl -sI https://registry.k8s.io    | head -1
curl -sI https://docs.tigera.io     | head -1
curl -sI https://get.helm.sh        | head -1
curl -sI https://quay.io            | head -1
curl -sI https://registry-1.docker.io/v2/ | head -1
sudo dnf makecache
```

- `https://pkgs.k8s.io` kubeadm 패키지 저장소
- `https://registry.k8s.io` 컨트롤 플레인 이미지
- `https://docs.tigera.io` Calico Helm repo
- `https://get.helm.sh` Helm 바이너리
- `https://quay.io` 이미지 레지스트리
- `https://registry-1.docker.io/v2/` 401이 와도 도달 성공
- `sudo dnf makecache` Rocky 저장소 접근

1.4 스냅샷(선택)

작업이 잘못되면 되돌릴 수 있도록 **단계 끝마다 스냅샷**을 만든다.

2. kubeadm 설치 요구사항

2.1 OS 버전 확인

- 지원되지 않는 커널 버전 오류 방지
- 필요한 기능을 커널 버전이 지원하는지 확인 리눅스 커널 버전 요구 사항

```
cat /etc/os-release | head -3
uname -r
```

2.2 모든 노드에서 MAC 주소와 product_uuid가 고유한지 확인

- MAC 주소 확인: `ip link | grep link/ether`
- product_uuid 확인: `sudo cat /sys/class/dmi/id/product_uuid`
- 호스트네임 확인: `hostname`
- 일부 가상 머신이 동일한 값을 가질 위험이 있어 확인
  - 쿠버네티스가 이러한 값을 사용하여 클러스터의 노드를 고유 식별하기 때문

2.3 네트워크 어댑터 확인

```
ip -br a
ip route | grep default
```

- 네트워크 어댑터가 두 개 이상이고, 노드 통신 어댑터가 따로 있는 경우 클러스터 주소가 적절한 어댑터를 통해 이동하도록 IP 경로를 추가하는 것이 좋다.
  - `kubeadm-config.yaml` 수정

```
apiVersion: kubeadm.k8s.io/v1beta4
kind: InitConfiguration
localAPIEndpoint:
  advertiseAddress: 192.168.2.101   # 노드 통신 어댑터(NIC) IP
```

  - 수정 후 `kubeadm init --config kubeadm-config.yaml`

실제 확인



- 고정 IP 적용 → **proto static**

2.4 필수 포트 확인

- 아래 포트들이 사용 중이 아닌지 ss로 확인한다. `sudo ss -tlnp`

컨트롤 플레인

프로토콜	방향	포트 범위	용도	사용 주체
TCP	인바운드	6443	쿠버네티스 API 서버	전부
TCP	인바운드	**2379-2380**	**etcd 서버 클라이언트 API**	kube-apiserver, etcd(2380은 etcd 서버끼리 통신용)
TCP	인바운드	**10250**	**Kubelet API**	Self, 컨트롤 플레인
TCP	인바운드	10259	kube-scheduler	Self
TCP	인바운드	10257	kube-controller-manager	Self

워커 노드

프로토콜	방향	포트 범위	용도	사용 주체

TCP	인바운드	10250	Kubelet API	Self, 컨트롤 플레인
TCP	인바운드	30000-32767	NodePort 서비스	전부

- **CNI가 쓰는 포트**도 확인해야 한다.
  - 현재 과제 스펙에 필요한 포트는 179 하나이다.

용도	포트
Calico BGP (노드 간 경로 교환)	**TCP 179**, 모든 노드, 양방향
IP-in-IP 캡슐화	프로토콜 번호 4
VXLAN 캡슐화	UDP 4789
Typha(대규모 클러스터용 캐시)	TCP 5473 (Typha를 쓸 때만)

- CNI 요구사항에 따라 firewalld 같은 iptables 관리자를 비활성화해야 할 수도 있다.
  → 근데 애초에 쿠버네티스 포트만 방화벽 포트를 다 열기보단 firewalld를 비활성화

```
sudo systemctl disable --now firewalld
systemctl is-active firewalld
```

2.5 스왑 구성

- 스왑을 영구적으로 비활성화한다.

```
swapon --show                       # 지금 켜진 swap과 그 위치, 출력 없으면 비활성화 상태
grep -n swap /etc/fstab             # fstab에 swap 줄이 있는지
sudo vi /etc/fstab
#/dev/mapper/rl-swap    none                    swap    defaults        0 0
# 위처럼 직접 주석 처리
sudo reboot
free -h # 스왑 0인지
```

- 임시 비활성화(즉시 반영, 비영구): `sudo swapoff -a`

> kubelet은 swap이 켜진 노드에서 기본적으로 시작을 거부한다. swap을 쓰면 파드의 memory requests/limits와 실제 사용량이 어긋나 스케줄링 예측이 흐려지기 때문.

2.6 SELinux permissive 설정

Red Hat 계열에선 컨테이너가 호스트 파일시스템에 접근하기 위해 SELinux를 permissive로 설정한다.
마찬가지로 vim을 사용해도 된다.

```
sudo setenforce 0
getenforce                              # Permissive
sudo vi /etc/selinux/config
```

```
4 grep ^SELINUX= /etc/selinux/config      # SELINUX=permissive
```

2.7 IPv4 포워딩

기본적으로 리눅스 커널은 인터페이스 간 IPv4 패킷 라우팅을 허용하지 않는다.

```
# 설정에 필요한 sysctl 파라미터이며, 재부팅 후에도 유지된다.
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.ipv4.ip_forward = 1
EOF
# 재부팅 없이 sysctl 파라미터 적용
sudo sysctl --system
sysctl net.ipv4.ip_forward # 1인지 확인
```

3. 컨테이너 런타임 설치

containerd를 설치한다.

containerd 설정은 노드마다 직접 해야 한다.

1. Docker 저장소를 추가하고 `containerd.io` 만 설치
   a. 저장소 추가 후 설치

```
sudo dnf install -y dnf-plugins-core python3-dnf-plugin-versionlock
sudo dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo

dnf list --showduplicates containerd.io | tail
sudo dnf install -y containerd.io-2.3.6
sudo dnf versionlock add containerd.io
containerd --version
```

   i. config-manager를 설치 후 containerd를 설치한다.

   ii. 모든 노드에서 버전이 같은지 확인한다.(버전 고정이 됐는지)

2. 설정 파일 새로 생성 및 부팅 시 자동 시작

```
sudo sh -c 'containerd config default > /etc/containerd/config.toml'
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
# sudo vi /etc/containerd/config.toml도 가능하다.

grep -n SystemdCgroup /etc/containerd/config.toml
sudo grep -n disabled_plugins /etc/containerd/config.toml
sudo systemctl enable containerd

sudo systemctl restart containerd
```

   a. /SystemdCgroup 검색 → false를 true로 수정한다.

   b. grep으로 true가 나오는지, cri가 없는지 확인 후 재시작한다.

3. 설정 확인

```
systemctl is-active containerd
systemctl is-enabled containerd
containerd config dump | grep SystemdCgroup
```

4. 재부팅 후 재확인(선택)

```
sudo reboot
```

```
systemctl is-active containerd
sysctl net.ipv4.ip_forward
swapon --show                    # 출력 없어야 함
```

4. kubeadm, kubelet, kubectl 설치

- 버전은 1.36.4로 고정한다.

> - `kubeadm` : 클러스터를 부트스트랩하는 명령
> - `kubelet` : 클러스터의 모든 머신에서 실행되는 파드와 컨테이너 시작과 같은 작업을 수행하는 컴포넌트
> - `kubectl` : 클러스터와 통신하기 위한 커맨드 라인 유틸리티

4.1 yum repository 추가

```
cat <<EOF | sudo tee /etc/yum.repos.d/kubernetes.repo
[kubernetes]
name=Kubernetes
baseurl=https://pkgs.k8s.io/core:/stable:/v1.36/rpm/
enabled=1
gpgcheck=1
gpgkey=https://pkgs.k8s.io/core:/stable:/v1.36/rpm/repodata/repomd.xml.key
exclude=kubelet kubeadm kubectl cri-tools kubernetes-cni
EOF
```

- exclude=kubelet kubeadm kubectl cri-tools kubernetes-cni: 업데이트에서 제외

4.2 버전 확인 후 설치

```
dnf list --showduplicates kubeadm --disableexcludes=kubernetes | tail
sudo dnf install -y kubelet-1.36.4 kubeadm-1.36.4 kubectl-1.36.4 --disableexcludes=kubernetes
sudo systemctl enable --now kubelet
```

4.3 버전 고정 확인

cri-tools(crictl)는 선택이다. `kubeadm init` preflight에서 crictl 관련 경고나 에러가 나오면 설치해도 된다.

```
kubeadm version -o short                # v1.36.4
rpm -q kubeadm kubelet kubectl
# 버전 고정 확인: kube* 가 업데이트 대상에서 빠지는지
grep ^exclude /etc/yum.repos.d/kubernetes.repo
```

---

5. 클러스터 세팅

> ⚠ **init 전 완료할 것**

1. kubeadm config 검증 `kubeadm config validate --config ~/kubeadm-config.yaml`
   a. **Pod CIDR, Service CIDR, 호스트 대역 안 겹치는지**
2. swap 비활성화
3. 방화벽 비활성화
4. SELinux permissive
5. IPv4 포워딩 활성화
6. 포트 점유 없는지

5.1 컨트롤플레인 구성: kubeadm init

~/kubeadm-config.yaml

```yaml
apiVersion: kubeadm.k8s.io/v1beta4
kind: InitConfiguration
localAPIEndpoint:
  advertiseAddress: 192.168.2.101
  bindPort: 6443
---
apiVersion: kubeadm.k8s.io/v1beta4
kind: ClusterConfiguration
kubernetesVersion: v1.36.4
networking:
  podSubnet: 172.16.0.0/22
  serviceSubnet: 10.96.0.0/24
---
apiVersion: kubelet.config.k8s.io/v1beta1
kind: KubeletConfiguration
cgroupDriver: systemd
```

```
kubeadm config validate --config ~/kubeadm-config.yaml
kubeadm init --config ~/kubeadm-config.yaml | tee ~/kubeadm-init.log
# tee는 선택
# chmod 600 ~/kubeadm-init.log
# 토큰이 있으므로 권한 설정
```

kubectl 접근 영구화

- root의 경우

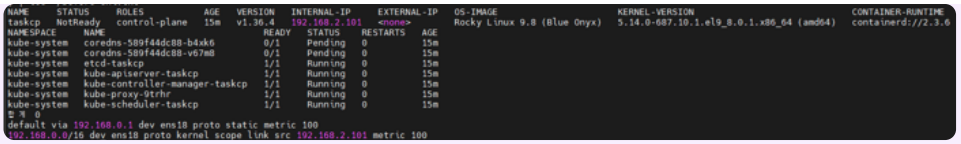
```
export KUBECONFIG=/etc/kubernetes/admin.conf
echo 'export KUBECONFIG=/etc/kubernetes/admin.conf' >> ~/.bashrc
source ~/.bashrc
```

- 일반 사용자의 경우

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

CNI 설치 이전 상태 확인

```
{
  kubectl get nodes -o wide
  kubectl get pods -A
  ls -l /etc/cni/net.d 2>&1
  ip route
} | tee ~/before-cni.txt # tee는 선택(단순 비교용)
```



- 노드 NotReady
- coredns Pending

5.2 네트워크 서비스: CNI

(1) NetworkManager 설정

Networkmanager가 Calico의 veth 쌍이 붙는 기본 ns의 라우팅 테이블을 건드려서 Calico 에이전트의 경로 설정을 방해할 수 있기에 Calico 설치 전에 이루어져야 하는 작업이다. **모든 노드에 필수**

```
sudo tee /etc/NetworkManager/conf.d/calico.conf <<'EOF'
[keyfile]
unmanaged-devices=interface-name:cali*;interface-name:tunl*;interface-name:vxlan.calico;interface-name:vxlan-v6.calico;interface-name:wireguard.cali;interface-name:wg-v6.cali
EOF
sudo systemctl reload NetworkManager # 확인
```

(2) Helm 설치

Helm 개념: **차트**(템플릿 + 기본 values.yaml 패키지), **values**(바꿀 값), **release**(설치된 결과)

Calico 공식 문서대로 Helm 3을 설치한다.

```
HELM_VERSION=v3.22.0
cd /tmp
curl -fsSLO https://get.helm.sh/helm-${HELM_VERSION}-linux-amd64.tar.gz
sha256sum helm-${HELM_VERSION}-linux-amd64.tar.gz
tar -zxvf helm-${HELM_VERSION}-linux-amd64.tar.gz
install -m 0755 linux-amd64/helm /usr/local/bin/helm
helm version    # 확인
```

→ 출력된 해시가 릴리스 페이지의 Linux amd64 체크섬과 같은지 비교한다. Releases · helm/helm

(3) Calico 설치

- Calico Helm repo를 추가한다.

```
helm repo add projectcalico https://docs.tigera.io/calico/charts
```

- values.yaml 파일을 생성해 IPPool을 설정해 준다.

```
cat <<'EOF' > ~/values.yaml
installation:
  calicoNetwork:
    bgp: Enabled
    ipPools:
      - cidr: 172.16.0.0/22
        encapsulation: None
EOF
```

- encapsulation(`IPIP` , `VXLAN` , `IPIPCrossSubnet` , `VXLANCrossSubnet` , `None`)을 명시하지 않으면 기본값은 IPIP가 된다.

- `tigera-operator` 네임스페이스를 생성한다.

```
kubectl create namespace tigera-operator
```

- 쿠버네티스 1.36 이후 버전용 스크립트로 CRD를 설치한다.

```
helm template calico-crds projectcalico/projectcalico.org.v3 --version v3.33.0 --api-versions admissionregistration.k8s.io/v1/MutatingAdmissionPolicy | kubectl apply --server-side -f -
```

- 헬름차트로 Tigera Operator를 설치한다.

```
helm install calico projectcalico/tigera-operator --version v3.33.0 -f ~/values.yaml --namespace tigera-operator
```

- 파드 상태를 확인한다.

```
watch kubectl get pods -n calico-system
```

```
NAME                                       READY   STATUS    RESTARTS   AGE
calico-kube-controllers-56bb565fdd-t5kgd   1/1     Running   0          2m20s
calico-node-t5qbj                          1/1     Running   0          2m20s
calico-typha-c87b8f985-28sk2               1/1     Running   0          2m20s
calico-webhooks-7d655bcb48-bgcc5           1/1     Running   0          2m21s
calico-webhooks-7d655bcb48-dgz6z           1/1     Running   0          2m21s
goldmane-7c87b647d7-w5r4n                  1/1     Running   0          2m21s
whisker-66b8689dbc-pqvpm                   2/2     Running   0          76s
```

(4) 검증

워커 노드를 붙이기 전 아래 검증을 모두 통과해야 한다.

```
kubectl get tigerastatus
kubectl get pods -n calico-system
kubectl get nodes -o wide
kubectl get pods -A -o wide
ip -br a
ip route
kubectl get ippools
```

항목	통과 기준
tigerastatus	전부 Available=True, Progressing/Degraded=False
노드	Ready
coredns	Running
ens18	UP, IP 유지
IPPool	172.16.0.0/22, VXLAN Never, IPIP Never, NAT true
터널 장치	`tunl0` , `vxlan.calico` 없음
경로	`blackhole <노드 블록>/26 proto bird` + Pod별 `/32 dev cali*`

(5) calicoctl 설치(선택)

원하는 경로로 이동한 후 클러스터와 일치하는 버전의 calicoctl 바이너리를 설치한다.

```
cd /usr/local/bin
curl -L
https://github.com/projectcalico/calico/releases/download/v3.33.0/calic
octl-linux-amd64 -o calicoctl
chmod +x ./calicoctl
calicoctl version
```

5.3 워커 노드 조인

(1) 노드별 사전 점검

작업	CP	워커
VM 세팅(hostname, 고정 IP, hosts), MAC/product_uuid 고유성	○	○
swap 해제, SELinux permissive, firewalld 비활성, `ip_forward=1`	○	○
containerd 설치, `SystemdCgroup = true`, enable	○	○
kubeadm / kubelet / kubectl 1.36.4 + exclude 고정	○	○ (kubectl은 선택)
NetworkManager `calico.conf`	○ (Calico 설치 전)	○ (join 전)
`kubeadm init`, Helm, Calico, calicoctl	○	×
`kubeadm join`	×	○
kubectl 작업(apply, get, describe)	○	× (kubeconfig 없음)
이미지 import(`ctr`)	×	○

⚠ join 토큰

kubeadm init이 만든 토큰은 TTL 24시간이라 워커 join이 하루를 넘기면 새 토큰이 필요
조인 후 kubelet은 토큰이 아니라 자기 클라이언트 인증서로 통신
→ 부트스트랩 토큰은 **조인 시점의 입장권** → join 완료됐다면 토큰 만료는 영향 없다.

- 토큰이 하는 일

◦ 클러스터 정보를 받아오고 CA 해시로 검증

◦ kubelet이 토큰으로 임시 인증 후 인증서 서명 요청

→ CP가 자동 서명

→ kubelet이 발급받은 인증서로 API 서버 접속

- 만료되는 이유

: 토큰을 가진 누구나 인증된 노드를 클러스터에 추가 가능 → **노드 추가 권한 제어**

→ 채팅이나 문서에 넣지 않는다.

- 확인 및 재발급

```
kubeadm token list                    # 남은 TTL / 만료 시각 확인
kubeadm token create --print-join-command         # 새 토큰 + CA 해시가 든 완성된 join 명령 출력(CP에서)
-> 출력된 것을 그대로 워커노드 root에서 실행
```

(2) join 실행

CP가 출력한 명령을 그대로 워커에서 실행한다.

```
kubeadm join 192.168.2.101:6443 --token <토큰> --discovery-token-ca-cert-hash sha256:<해시>
```

(3) join 검증

- CP 검증

```
kubectl get nodes -o wide                       # 워커 2대 Ready
kubectl get pods -n calico-system -o wide     # calico-node가 노드마다 1개
kubectl get tigerastatus
kubectl get pods -A -o wide
```

- 각 노드

```
systemctl is-active containerd kubelet
ip route | grep bird
sudo calicoctl node status # 선택
```

5.4 클러스터 서비스 상태 점검

각 서비스를 어디서 확인할 수 있는지 정리한 표이다.

서비스	역할	실행 형태 / 위치	확인
containerd	컨테이너 실행, 이미지 관리 (CRI)	systemd, 모든 노드	`systemctl is-active containerd`
kubelet	노드에서 파드 실행, probe 수행	systemd, 모든 노드	`systemctl is-active kubelet`

kube-apiserver, etcd, kube-scheduler, kube-controller-manager	컨트롤 플레인	Static 파드, CP	`kubectl get pods -n kube-system -o wide`
kube-proxy	Service 규칙을 iptables로 작성	DaemonSet, 모든 노드	`kubectl -n kube-system get ds kube-proxy`, `sudo iptables-save -t nat | grep KUBE-SVC`
CoreDNS	서비스 이름 DNS	Deployment, kube-system	`kubectl get svc -n kube-system kube-dns`
tigera-operator	Calico 설치·관리	Deployment, tigera-operator ns	`kubectl get pods -n tigera-operator`
calico-node	노드별 BGP, 경로·정책	DaemonSet, calico-system	`kubectl get pods -n calico-system -o wide`, `ip route | grep bird`
Helm	차트 설치 도구	CP의 바이너리	`helm list -A`

한번에 점검하기

```
systemctl is-active containerd kubelet
kubectl get nodes
kubectl get pods -A
kubectl get tigerastatus
ip route | grep bird
kubeadm token list
```

5.5 이미지 반입

사내 레지스트리가 없으면 이미지를 tar로 옮겨 워커의 containerd에 직접 넣는다.

(원래는 Harbor 같은 레지스트리에 push하고 노드가 pull)

- 이미지 빌드

```
FROM nginx:stable
COPY index.html /usr/share/nginx/html/index.html
COPY ready.html /usr/share/nginx/html/ready.html
EXPOSE 80
```

```
chmod 644 */index.html */ready.html           # nginx 워커가 읽으려면 권한 필요
docker build -t os-intro-rhel:1.0 ./rhel
docker build -t os-intro-rocky:1.0 ./rocky
docker build -t os-intro-ubuntu:1.0 ./ubuntu
docker images | grep os-intro
docker run -d --rm --name t-rhel -p 8084:80 os-intro-rhel:1.0 && curl -v
localhost:8084/    # 3개 모두 동작 확인
```

- scp와 import

```
cd ~/os-intro-tar
docker save os-intro-rhel:1.0 -o os-intro-rhel.tar
scp *.tar root@192.168.2.102:~/
scp *.tar root@192.168.2.103:~/

# 워커에서
sudo ctr -n k8s.io images import os-intro-rhel.tar
sudo ctr -n k8s.io images ls | grep os-intro
```

`-n k8s.io`가 핵심이다. kubelet은 `k8s.io` 네임스페이스만 보기 때문에 다른 곳에 넣으면 파드가 이미지를 찾지 못한다.

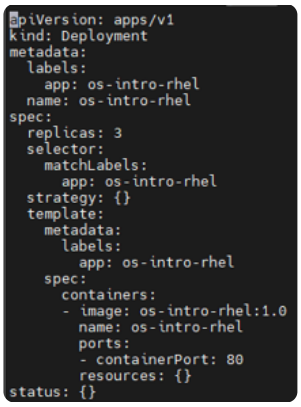
→ **네임스페이스 지정 필요**

이미지는 **파드가 배치될 모든 워커**에 넣어야 한다.

6. 클러스터 오브젝트

6.1 Deployment와 probe

```
kubectl create deployment os-intro-rhel --image=os-intro-rhel:1.0 --
replicas=3 --port=80 \
  --dry-run=client -o yaml > os-intro-rhel.yaml
```



```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: os-intro-rhel
  name: os-intro-rhel
spec:
  replicas: 3
  selector:
    matchLabels:
      app: os-intro-rhel
  strategy: {}
  template:
    metadata:
      labels:
        app: os-intro-rhel
    spec:
      containers:
      - image: os-intro-rhel:1.0
        name: os-intro-rhel
        ports:
        - containerPort: 80
        resources: {}
status: {}
```

default

생성된 yaml 파일에 `imagePullPolicy` 와 두 **probe**를 추가한다.
이때 들여쓰기는 탭이 아닌 스페이스를 사용한다.

```
        image: os-intro-rhel:1.0
        imagePullPolicy: IfNotPresent
        ports:
        - containerPort: 80
        livenessProbe:
          httpGet:
            path: /index.html
            port: 80
          initialDelaySeconds: 3
          periodSeconds: 5
        readinessProbe:
          httpGet:
            path: /ready.html
            port: 80
          initialDelaySeconds: 3
          periodSeconds: 5
```

- **liveness**: 실패하면 kubelet이 컨테이너를 죽이고 새로 만든다. → **컨테이너 ID 바뀜**
- **readiness**: 실패하면 파드는 유지되고 Endpoint 목록(트래픽 대상)에서만 빠진다.
  복구되면 자동 복귀.

6.2 Service

- NodePort

```
echo '---' >> os-intro-rhel.yaml
kubectl create service nodeport os-intro-rhel --tcp=80:80 --node-port=30081 \
  --dry-run=client -o yaml >> os-intro-rhel.yaml
kubectl apply --dry-run=server -f os-intro-rhel.yaml     # 문법·서버 검증
kubectl apply -f os-intro-rhel.yaml                # 워커가 Ready일 때 실제 적용
kubectl get deploy,svc,pods -o wide
```

- 최종 yaml 파일 검증

```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: os-intro-rhel
  name: os-intro-rhel
spec:
  replicas: 3
  selector:
```

```
    matchLabels:
      app: os-intro-rhel
  template:
    metadata:
      labels:
        app: os-intro-rhel
    spec:
      containers:
      - name: os-intro-rhel
        image: os-intro-rhel:1.0
        imagePullPolicy: IfNotPresent
        ports:
        - containerPort: 80
        livenessProbe:
          httpGet:
            path: /index.html
            port: 80
          initialDelaySeconds: 3
          periodSeconds: 5
        readinessProbe:
          httpGet:
            path: /ready.html
            port: 80
          initialDelaySeconds: 3
          periodSeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  labels:
    app: os-intro-rhel
  name: os-intro-rhel
spec:
  type: NodePort
  selector:
    app: os-intro-rhel
  ports:
  - name: 80-80
    port: 80
    targetPort: 80
    protocol: TCP
    nodePort: 30081
```

- sed 문법 사용

```
sed 's/rhel/rocky/g; s/30081/30082/'  os-intro-rhel.yaml > os-intro-rocky.yaml
sed 's/rhel/ubuntu/g; s/30081/30083/' os-intro-rhel.yaml > os-intro-ubuntu.yaml
kubectl apply -f os-intro-rocky.yaml -f os-intro-ubuntu.yaml
```

- replicas 확장

```
kubectl scale deployment os-intro-rhel --replicas=5
# 또는 yaml의 replicas 수정 후 apply
kubectl get pods -o wide
```

- LoadBalancer
  - `type: LoadBalancer`는 ClusterIP + NodePort + 외부 LB 요청
  - 외부 LB가 없으면 EXTERNAL-IP가 pending
  - 외부 LB는 CCM이나 MetalLB 같은 구현체가 비동기로 만듦

6.3 Ingress Controller

Ingress 리소스는 **선언**이고, 처리하는 것은 **컨트롤러**(NGINX 파드)
→ 컨트롤러가 없으면 실행X

- ~/ingress-values.yaml 작성

```yaml
controller:
  service:
    type: NodePort
    externalTrafficPolicy: Cluster
    httpPort:
      nodePort: 30080
    httpsPort:
      nodePort: 30443
```

  - 차트 기본값은 `type: LoadBalancer`, `externalTrafficPolicy: Local`, nodePort 빈 값(랜덤)
    - `Cluster` : 어느 노드로 들어와도 컨트롤러 파드가 있는 노드로 전달(NodePort 경로에서 SNAT, 클라이언트 IP가 노드 IP로 바뀜)
    - `Local` : 파드가 있는 노드만 응답(소스 IP 보존)

- **helm 차트를 사용한 설치**

```bash
helm show chart oci://ghcr.io/nginx/charts/nginx-ingress --version 2.7.3 | grep -E '^(version|appVersion):'
helm template nginx-ingress oci://ghcr.io/nginx/charts/nginx-ingress --version 2.7.3 \
  -n nginx-ingress -f ~/ingress-values.yaml | grep -n -E 'type: (NodePort|LoadBalancer)|nodePort|externalTrafficPolicy'

# 설치+적용
helm install nginx-ingress oci://ghcr.io/nginx/charts/nginx-ingress --version 2.7.3 \
  -n nginx-ingress --create-namespace -f ~/ingress-values.yaml

helm list -n nginx-ingress
helm get values nginx-ingress -n nginx-ingress
kubectl -n nginx-ingress get pods,svc -o wide     # NodePort, 80:30080 443:30443
kubectl get ingressclasses                        # nginx
```

  - `upgrade`에서는 `--version`을 반드시 고정

6.4 Ingress 리소스: 호스트 기반 라우팅

같은 IP:포트로 들어온 HTTP 요청을 **Host 헤더**로 구분
NodePort는 패킷을 컨트롤러 Service로 보낼 뿐이고, Host로 나누는 것은 컨트롤러(NGINX)의 `server_name` 매칭

- ~/ingress.yaml 작성

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: os-intro
spec:
  ingressClassName: nginx
  rules:
  - host: rhel.[redacted].net # server_name 매칭에 사용
    http:
      paths:
      - path: /
        pathType: Prefix
```

```
        backend:
          service:
            name: os-intro-rhel # upstream
            port:
              number: 80
  - host: rocky.[redacted].net
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: os-intro-rocky
            port:
              number: 80
  - host: ubuntu.[redacted].net
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: os-intro-ubuntu
            port:
              number: 80
```

  - `ingressClassName: nginx`는 반드시 지정

- 적용

```
kubectl apply -f ~/ingress.yaml
kubectl get ingress
kubectl describe ingress os-intro        # Rules에서 host별 backend(파드 IP) 확인
```

- 접속 PC의 hosts 등록(Windows: `C:\Windows\System32\drivers\etc\hosts`, 관리자 권한)

```
192.168.2.102[redacted]
192.168.2.102[redacted]
192.168.2.102[redacted]
```

- 브라우저 확인 [redacted]

> ① 클라이언트: hosts/DNS가 rhel[redacted].net → 192.168.2.102 (이름 → IP)
>
> ② TCP 연결: 192.168.2.102:30080
>
> ③ HTTP 요청: GET / , Host: rhel[redacted].net:30080 (이름이 Host 헤더에 실려 감)
>
> ④ kube-proxy(NodePort) → 컨트롤러 Service → **컨트롤러 파드 NGINX Host를 server_name과 비교** → 일치하는 server 블록 → upstream = os-intro-rhel 파드들 → 응답

6.5 TLS와 http→https 리다이렉트

- 제공받은 인증서(crt와 key)를 CP에 반입한다.

```
scp <[redacted].crt> <[redacted].key> <계정>@192.168.2.101:~/
```

- 제대로 옮겨졌는지 PC와 VM 양쪽에서 확인한다.

```
# PC
```

```
(Get-FileHash .\all.crt.pem -Algorithm SHA256).Hash.ToLower()
(Get-FileHash .\key.pem     -Algorithm SHA256).Hash.ToLower()

# VM
sha256sum <옮긴 인증서파일> <옮긴 키파일>
```

- CP에서 key 파일 권한을 600으로 설정한다.

```
chmod 600 <      .key>
```

- 제공받은 공인 인증서를 Secret으로 만들어 Ingress에 연결한다.

```
openssl x509 -in      .crt -noout -subject -issuer -dates -ext subjectAltName
# SAN에 *.      .net 또는 세 호스트

openssl x509 -in      .crt -noout -pubkey | openssl sha256
openssl pkey -in      .key -pubout | openssl sha256
# 두 해시값 동일한지

kubectl create secret tls      -tls --cert=      .crt --key=      .key
kubectl describe secret      -tls
# tls.crt, tls.key, type kubernetes.io/tls
```

  - Secret은 Ingress와 같은 네임스페이스에 둔다.

- `~/ingress.yaml`의 `spec`에 tls 추가

```
spec:
  ingressClassName: nginx
  tls:
  - hosts:
    - rhel.      .net
    - rocky.      .net
    - ubuntu.      .net
    secretName:      -tls
  rules:
  ...
```

  - 적용 후 확인

```
kubectl apply -f ~/ingress.yaml
kubectl describe ingress os-intro        # TLS:      -tls 확인

# 1) HTTP → 리다이렉트 되는지 확인
curl -I --resolve rhel.      .net:30080:192.168.2.102 http://rhel.      .net:30080/

# 2) HTTPS 접속(포트 30443). 자체 서명은 --cacert      .crt 추가, 공인 인증서는 옵션 없이
curl -sv --resolve rhel.      .net:30443:192.168.2.102 https://rhel.      .net:30443/ -
o /dev/null 2>&1 \
  | grep -E 'SSL certificate verify|subject:|issuer:|expire date|HTTP/'
```

- 공인 인증서가 없을 땐 사설 인증서를 생성한다.

```
openssl req -x509 -nodes -newkey rsa:2048 -days 365 \
  -keyout      -selfsigned.key -out      -selfsigned.crt \
  -subj "/CN=*.      .net" \
  -addext "subjectAltName=DNS:rhel.      .net,DNS:rocky.      .net,DNS:ubuntu.      .net"
chmod 600      -selfsigned.key

openssl x509 -in      -selfsigned.crt -noout -subject -issuer -dates -ext
subjectAltName
# SAN에 세 호스트, subject와 issuer가 같으면 자체 서명

kubectl create secret tls      -tls --cert=      -selfsigned.crt --key=      -
selfsigned.key
```

7. etcd 백업 및 복구

7.1 백업

(1) etcdctl/etcdutl 설치

호스트에는 두 도구가 없다. 복구 중에는 etcd 파드가 내려가기에 복구 전에 설치해 둔다.

```
# 1) etcd 파드가 쓰는 이미지 확인
IMG=$(grep -m1 'image:' /etc/kubernetes/manifests/etcd.yaml | awk '{print $2}')
echo "$IMG"

ctr -n k8s.io images ls -q | grep etcd               # 노드에 이미 있는지
# 없으면 레지스트리에서 받는다(registry.k8s.io 접속 필요, 1.3 참고)
# ctr -n k8s.io images pull "$IMG"

# 2) 이미지의 루트 파일시스템을 마운트해서 바이너리만 복사
mkdir -p /mnt/etcd-img
ctr -n k8s.io images mount "$IMG" /mnt/etcd-img
cp /mnt/etcd-img/usr/local/bin/etcdctl /mnt/etcd-img/usr/local/bin/etcdutl /usr/local/bin/
umount /mnt/etcd-img
etcdctl version; etcdutl version                     # 기대: 둘 다 3.6.8
```

(2) 환경변수 설정

```
cat > /root/etcd-env.sh <<'EOF'
export ETCDCTL_ENDPOINTS=https://127.0.0.1:2379
export ETCDCTL_CACERT=/etc/kubernetes/pki/etcd/ca.crt
export ETCDCTL_CERT=/etc/kubernetes/pki/etcd/server.crt
export ETCDCTL_KEY=/etc/kubernetes/pki/etcd/server.key
EOF

source /root/etcd-env.sh                             # 새 셸마다 실행
etcdctl endpoint health -w table
```

(3) 기존 상태 확인

스냅샷 찍기 전 상태를 확인한다.

```
kubectl create namespace etcd-lab
kubectl get ns,deploy,svc,ingress,secret -A
kubectl get nodes,pods -A -o wide
etcdctl get /registry/namespaces/etcd-lab --keys-only
etcdctl endpoint status -w table
```

(4) 스냅샷 저장과 검증, 복사

```
mkdir -p /root/etcd-backup && chmod 700 /root/etcd-backup
SNAP=/root/etcd-backup/snapshot-$(date +%Y%m%d-%H%M).db
etcdctl snapshot save $SNAP
etcdutl --write-out=table snapshot status $SNAP      # REVISION, TOTAL KEYS 확인

chmod 600 $SNAP
scp $SNAP root@192.168.2.102:/root/                  # VM 밖(다른 노드)에 복사
```

7.2 복구

API 서버가 도는 동안 etcd를 복구하지 않는다.

API 서버를 모두 멈추고, 복구한 뒤 restart 한다.

스냅샷은 **같은 major.minor의 etcd로 복구**하고 patch가 달라도 된다.

(연습) 변경사항 생성

```
kubectl delete namespace etcd-lab
kubectl get ns etcd-lab
```

(1) 컨트롤 플레인 중지

```
mkdir -p /root/manifests-orig /root/manifests-moved
cp -a /etc/kubernetes/manifests/. /root/manifests-orig/
mv /etc/kubernetes/manifests/*.yaml /root/manifests-moved/
sleep 60

pgrep -x etcd
pgrep -f kube-apiserver
```

- 매니페스트를 밖으로 옮기고, 원본 사본을 따로 남긴다.

(2) 복구 시작

```
SNAP=</root/etcd-backup/snapshot-20261014-1335.db>
grep -nE '\-\-(name|initial-cluster|initial-advertise-peer-urls)' /root/manifests-orig/etcd.yaml

etcdutl snapshot restore $SNAP \
  --data-dir /var/lib/etcd-restored \
  --name taskcp \
  --initial-cluster taskcp=https://192.168.2.101:2380 \
  --initial-advertise-peer-urls https://192.168.2.101:2380
ls -l /var/lib/etcd-restored/                          # member 디렉터리가 생김
```

(3) etcd.yaml의 호스트 경로만 수정

```
sed -i 's#^\(\s*path: \)/var/lib/etcd$#\1/var/lib/etcd-restored#' /root/manifests-moved/etcd.yaml
diff /root/manifests-orig/etcd.yaml /root/manifests-moved/etcd.yaml
```

(4) 컨트롤 플레인 복귀 확인

```
mv /root/manifests-moved/*.yaml /etc/kubernetes/manifests/
sleep 60

pgrep -x etcd
etcdctl endpoint health -w table
etcdctl endpoint status -w json | grep -o '"revision":[0-9]*'
kubectl get nodes
```

- revision은 etcd에 쓰기가 일어날 때마다 커지는 번호다,
- 복구하면 스냅샷 시점의 값에서 이어지고, API 서버가 다시 쓰기 시작하면 더 커진다. **그래서 스냅샷 값보다 작을 수 없다.**
- **스냅샷 이후 변경사항이 사라지고 이전 상태로 돌아왔는지 확인한다.**

(5) kubelet 재시작

`systemctl restart kubelet`

- scheduler와 controller-manager는 정적 파드라 1~4단계에서 매니페스트를 옮겼다 되돌리면서 이미 재시작되었다. kubelet은 systemd 서비스라 따로 재시작한다.

7.3 롤백

남겨둔 원래 데이터 `/var/lib/etcd`를 활용하여 롤백한다.

```
mv /etc/kubernetes/manifests/*.yaml /root/manifests-moved/ 2>/dev/null
cp -f /root/manifests-orig/*.yaml /etc/kubernetes/manifests/
```

---

8. 초기화와 재구축

순서는 워커 → CP다.

```
# CP: 워커 노드 제거
kubectl drain taskwk1 --ignore-daemonsets --delete-emptydir-data
kubectl delete node taskwk1          # taskwk2도 동일

# 각 워커
sudo kubeadm reset -f

# CP
sudo kubeadm reset -f
sudo rm -rf /var/lib/etcd-restored
# 7장에서 복구에 쓴 데이터 디렉터리(Secret 포함)
ls -A /var/lib/etcd
# 아무것도 안 나와야 함
```

- `kubeadm reset`이 정리하지 않는 것
  - CNI 설정 `/etc/cni/net.d`, iptables/IPVS 규칙, kubeconfig(`$HOME/.kube/config`)는 수동 정리
  - Calico 흔적: `cali*` 인터페이스, bird 경로

```
sudo rm -rf /etc/cni/net.d $HOME/.kube/config
sudo iptables -F && sudo iptables -t nat -F && sudo iptables -t mangle -F && sudo iptables -X
ip -br a | grep -E 'cali|tunl'; ip route | grep bird      # 남은 흔적 확인

pgrep -a -f 'calico-node|bird|felix'
iptables -S | grep -c cali
rm -rf /var/lib/cni /var/lib/calico

sudo reboot
```

정리한 뒤 이 문서의 **5.1장부터 순서대로** 다시 수행한다.

```
ip -br a | grep -E 'cali|tunl'; ip route | grep bird      # 아무것도 안 나와야 함
ls -A /var/lib/etcd                                       # 비어 있어야 함
```

```
systemctl is-enabled kubelet containerd                    # 둘 다 enabled (재구축 전 확인)
```